Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes

TD : IDM – Model Driven Architecture (MDA)

CIM, PIM, PDM, PSM et transformations de modèles

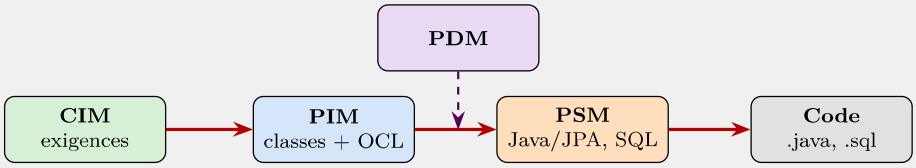
(Énoncé)

Pr.Aissam BERRAHOU
Année universitaire 2026–2027

Objectifs et consignes

1. Distinguer les modèles **CIM**, **PIM**, **PDM** et **PSM** et savoir ce que chacun contient.
2. Construire le PIM d'un système à partir de son CIM, puis le **projeter** sur une plateforme (Java/JPA, SQL).
3. **Classer** une transformation (sens, endogène/exogène, horizontale/verticale, M2M/M2T/T2M).
4. Écrire de petites transformations **par template** (XSLT), **par programmation** (Java) et **par modélisation** (ATL).

Le TD suit le **fil rouge du cours** : la **bibliothèque** (Adhérent, Livre, Emprunt). Les exercices 7 et 8 reprennent les deux « travaux à faire » du cours (jeux XML, machine à états du feu). Code couleur (comme dans le cours) :



Plan du TD :

Exercice	Thème	Notion principale
1	CIM, PIM, PSM ou code ?	Les modèles de MDA [Simple]
2	Du CIM au PIM : la bibliothèque	Cas d'utilisation, classes, OCL [Simple]
3	Ce PIM est-il vraiment indépendant ?	Indépendance vis-à-vis de la plateforme [Simple]
4	Projection PIM → PSM Java / JPA	PDM, règles de projection [Moyen]
5	Projection PIM → PSM relationnel	Règles R1 à R5, SQL [Moyen]
6	Classer des transformations	Taxonomie [Simple]
7	Transformation par template (XSLT)	M2T [Moyen]
8	Transformation par programmation (Java)	M2T, API de modèle [Moyen]
9	Transformation par modélisation (ATL)	M2M, règles déclaratives [Avancé]

Exercice 1 : CIM, PIM, PSM ou code ? [Simple]

a) Pour chaque élément de la bibliothèque, indiquer s'il appartient au **CIM**, au **PIM**, à un **PSM** ou au **code**. Un élément peut aussi être un **PDM** (description de la plateforme).

E1. L'exigence « un adhérent emprunte au plus 3 livres en même temps ».
E2. Le cas d'utilisation « Rendre un livre » et son acteur « Adhérent ».
E3. La classe `Livre` avec les attributs `isbn : String` et `titre : String`.
E4. La contrainte OCL `context Adherent inv : self.emprunts->size() <= 3`.
E5. La classe `Livre` stéréotypée «Entity» avec l'attribut «Id» `isbn : String`.
E6. La table `LIVRE(isbn VARCHAR(20) PK, titre VARCHAR(255), adherent_id BIGINT FK)`.
E7. Le tableau « String → VARCHAR(255), Integer → INTEGER, Date → DATE ».
E8. Le fichier `Livre.java` contenant `@Entity public class Livre { ...}`.
E9. Le profil UML « UML Profile for Java/EJB » (stéréotypes «Entity», «Id»...).
E10. Le diagramme d'états d'un `Livre` : `Disponible` → `Emprunté` → `Disponible`.

Élément	E1	E2	E3	E4	E5	E6	E7	E8	E9	E10
Modèle										

b) Le PIM et le PSM sont-ils au même niveau de la pyramide de l'OMG (M0 à M3) ? Justifier.
c) Pourquoi dit-on que le PIM est le modèle « le plus précieux » d'un projet MDA ?

Exercice 2 : Du CIM au PIM – la bibliothèque [Simple]

Le responsable de la bibliothèque de l'ENSIAS exprime son besoin :

> « Nos **adhérents** (nom, e-mail) peuvent **rechercher** un livre dans le catalogue, l'**emprunter** puis le **rendre**. Un livre est identifié par son **ISBN** ; il a un titre et un auteur. Chaque **emprunt** concerne un seul livre et un seul adhérent ; on note sa date de début et sa date de retour prévue. Un adhérent a **au plus 3 emprunts** en cours. Un emprunt dure **au plus 15 jours**. Le **bibliothécaire** enregistre les emprunts et les retours et gère le catalogue. »

a) [CIM] Dessiner le diagramme de cas d'utilisation.

b) [CIM] Écrire les exigences numérotées (REQ-1, REQ-2...) qui contiennent une règle de gestion.

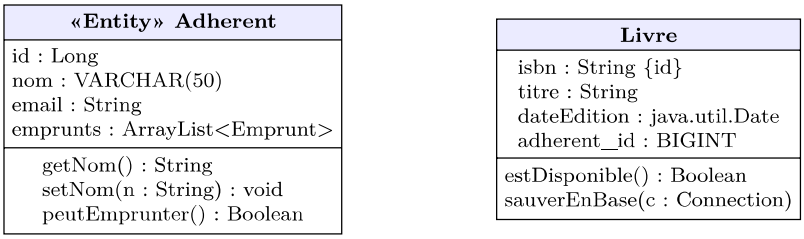
c) [PIM] Proposer le diagramme de classes (classes, attributs typés, associations, multiplicités). Utiliser uniquement les types `String`, `Integer`, `Boolean`, `Date`.

d) [PIM] Traduire chaque exigence de (b) en contrainte OCL. On suppose que `Emprunt` possède un attribut `duree : Integer` (en jours) et un attribut `rendu : Boolean`.

e) Quelle exigence relie le CIM au PIM ? Expliquer ce qu'apporte cette **traçabilité**.

Exercice 3 : Ce PIM est-il vraiment indépendant ? [Simple]

Un étudiant propose le « PIM » suivant pour la bibliothèque :



a) Relever **tous** les éléments qui dépendent d'une plateforme et dire de quelle plateforme il s'agit.

b) Proposer la version corrigée (un vrai PIM).

c) Où doivent aller les informations supprimées ?

Exercice 4 : Projection PIM → PSM Java / JPA [Moyen]

On utilise le PIM de l'exercice 2 (corrigé) et le PDM Java/JPA du cours :

Concept du PIM	Plateforme Java / JPA
Classe `C`	classe Java `C` annotée `@Entity`
Attribut marqué `{id}`	champ annoté `@Id`
Classe sans attribut `{id}`	ajouter `@Id @GeneratedValue private Long id;`
`String` / `Integer` / `Boolean` / `Date`	`String` / `int` / `boolean` / `java.time.LocalDate`
Attribut	champ `private` + accesseurs `get`/`set`
Extrémité de multiplicité `1`	champ de type `B` annoté `@ManyToOne(optional = false)`
Extrémité de multiplicité `0..*`	champ `List<B>` annoté `@OneToMany(mappedBy = ...)`
Opération	méthode `public` (corps à compléter à la main)

a) Dessiner le PSM Java (diagramme de classes stéréotypé) des trois classes.

b) Écrire le code Java **généré** pour la classe `Emprunt` (sans les accesseurs).

c) Compléter à la main le corps de `Adherent.peutEmprunter()` pour respecter REQ-1.

d) Le client demande une version **C# / .NET**. Qu'est-ce qui change dans la démarche ? Qu'est-ce qui ne change pas ?

Exercice 5 : Projection PIM → PSM relationnel [Moyen]

On complète le PIM de l'exercice 2 : un livre appartient à **plusieurs catégories** (`Categorie(libelle : String)`) et une catégorie contient plusieurs livres (association `Livre 0..*` — `0..* Categorie`). On applique les règles du cours, complétées par R5 :

R1	Toute classe `C` devient une table `C` (nom en majuscules).
R2	Tout attribut devient une colonne (`String` → `VARCHAR(255)`, `Integer` → `INTEGER`, `Boolean` → `BOOLEAN`, `Date` → `DATE`) ; un attribut `{id}` devient la clé primaire.
R3	Une classe sans attribut `{id}` reçoit une colonne `id BIGINT` clé primaire.
R4	Une association `1` (classe A) — `*` (classe B) ajoute dans la table B une colonne `a_<clé de A> NOT NULL`, clé étrangère vers A.
R5	Une association `*` (A) — `*` (B) devient une **table de jointure** `A_B` dont la clé primaire est formée des deux clés étrangères.

a) Donner le PSM relationnel (tables, colonnes, clés) en indiquant pour chaque élément la règle appliquée.

b) Écrire le script SQL (`CREATE TABLE`) correspondant.

c) La contrainte OCL `quota` (REQ-1) a-t-elle un équivalent dans ce PSM ? Que proposer ?

d) Cette transformation est-elle endogène ou exogène ? horizontale ou verticale ? M2M ou M2T ? Et la génération du script de la question b) ?

Exercice 6 : Classer des transformations [Simple]

Compléter le tableau. Sens : CIM→PIM, PIM→PIM, PIM→PSM…; Méta : **En**dogène / **Ex**ogène ; Direction : **H**orizontale / **V**erticale ; Nature : M2M, M2T ou T2M.

N°	Transformation	Sens	Méta	Dir.	Nature
T1	À partir des exigences SysML, l'analyste construit le diagramme de classes				
T2	On ajoute la classe `Emprunt` entre `Adherent` et `Livre`				
T3	On renomme `Adherent` en `Membre` dans tout le PIM				
T4	Le PIM est traduit en schéma relationnel (exercice 5)				
T5	Le PSM Java produit les fichiers `.java`				
T6	Le script SQL d'une ancienne base est lu pour construire un modèle relationnel				
T7	Le modèle relationnel obtenu en T6 est remonté en diagramme de classes				
T8	Un PSM .NET est converti en PSM Java				
T9	Les PIM « bibliothèque » et « médiathèque » sont fusionnés en un seul PIM				
T10	`article.xml` est transformé en page HTML par XSLT				

Exercice 7 : Transformation par template (XSLT) [Moyen]

Partie A – Lire un template. On reprend le template `article_txt.xsl` du cours :

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="text" encoding="UTF-8"/>
  <xsl:template match="/">
    <xsl:text>Article - </xsl:text>
    <xsl:value-of select="/Article/Title"/>
    <xsl:text>&#10;Auteurs :&#10;</xsl:text>
    <xsl:for-each select="/Article/Authors/Author">
      <xsl:text> - </xsl:text><xsl:value-of select="."/><xsl:text>&#10;</xsl:text>
    </xsl:for-each>
  </xsl:template>
</xsl:stylesheet>
```

A1. Donner le texte produit pour le document : `<Article><Title>Design Patterns</Title><Authors><Author>E. Gamma</Author> <Author>R. Helm</Author><Author>R. Johnson</Author></Authors></Article>`.

A2. Dans ce template, repérer le **texte fixe**, les **trous** et la **boucle**. Que représente `
` ?

Partie B – Le travail à faire du cours. Écrire `jeux.xsl` qui transforme `jeux.xml` (cours) en page HTML contenant un tableau `N° | Nom | Type | Prix`. Puis : (B2) trier les lignes par nom ; (B3) colorer en vert la ligne d'un jeu « Gratuit ».

Partie C – Vers la plateforme SQL. Écrire `livres_sql.xsl` qui transforme le document ci-dessous en script d'insertion SQL pour la table `LIVRE` de l'exercice 5.

```
<bibliotheque nom="Bibliothèque ENSIAS">
  <livre isbn="111"><titre>Le Petit Prince</titre><auteur>Saint-Exupéry</auteur></livre>
  <livre isbn="222"><titre>La Peste</titre><auteur>Camus</auteur></livre>
</bibliotheque>
```

Résultat attendu : `INSERT INTO LIVRE (isbn, titre, auteur) VALUES ('111', 'Le Petit Prince', 'Saint-Exupéry');` (une ligne par livre).

Exercice 8 : Transformation par programmation (Java) [Moyen]

Le méta-modèle (simplifié) des machines à états est : `StateMachine(name)` contient des `State(name)` et des `Transition(event)` ; une transition a une `source` et une `target` (des `State`) ; `StateMachine` a une référence `initial` vers un `State`. EMF génère l'API Java : `getName()`, `getStates()`, `getTransitions()`, `getInitial()`, `getSource()`, `getTarget()`, `getEvent()`.

On considère le modèle du **feu piéton** : `Rouge` (initial) $\xrightarrow{\text{bouton}}$ `Vert` $\xrightarrow{\text{after(15 s)}}$ `Clignotant` $\xrightarrow{\text{after(5 s)}}$ `Rouge`.

a) Donner le document XML attendu, au même format que l'exemple du cours (feu tricolore).

b) Compléter la méthode suivante :

```
public static String versXml(StateMachine sm) {
  StringBuilder sb = new StringBuilder();
  sb.append("<stateMachine name=\"" + ______ + "\">\n");
  for (State s : ______)
    sb.append(" <state name=\"" + s.getName() + "\""
        + (______ ? " initial=\"true\"" : "") + "/>\n");
  for (Transition t : sm.getTransitions())
    sb.append(" <transition from=\"" + ______ + "\" to=\"" + ______
```

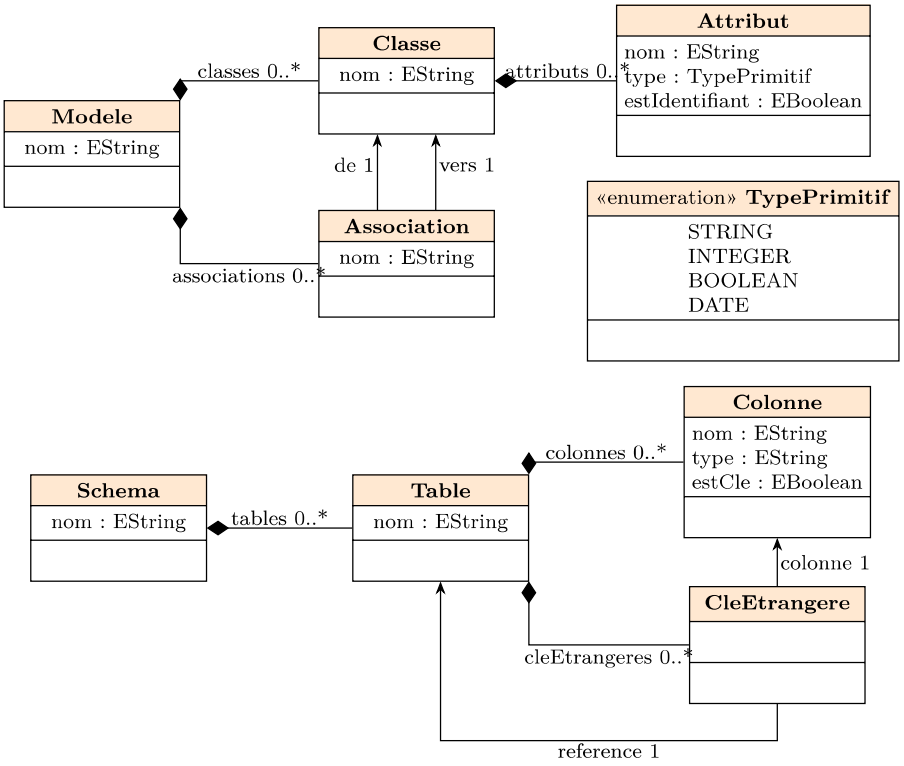
```
        + "\" event=\"" + t.getEvent() + "\"/>\n");
  return sb.append(______).toString();
}
```

c) Cette transformation est-elle M2M ou M2T ? Quelle autre approche du cours conviendrait mieux ? Pourquoi ?

d) On veut maintenant **relire** un tel fichier XML pour reconstruire le modèle. Comment s'appelle ce type de transformation ? Citer une utilité.

Exercice 9 : Transformation par modélisation (ATL) [Avancé]

Les deux méta-modèles utilisés par la transformation sont :



Dans le méta-modèle `Pim`, une `Association` relie une classe `de` (côté `1`) à une classe `vers` (côté `*`). Le cours donne les règles ATL de R1 et R2 :

```
module PIM2Relationnel;
create OUT : Rel from IN : Pim;

helper def : sqlType(t : Pim!TypePrimitif) : String =
  if t = #STRING then 'VARCHAR(255)' else if t = #INTEGER then 'INTEGER'
  else if t = #BOOLEAN then 'BOOLEAN' else 'DATE' endif endif endif;

rule Classe2Table { -- R1
  from c : Pim!Classe
  to t : Rel!Table ( nom <- c.nom.toUpper(), colonnes <- c.attributs )
}
rule Attribut2Colonne { -- R2
  from a : Pim!Attribut
```

```
  to k : Rel!Colonne ( nom <- a.nom, type <- thisModule.sqlType(a.type),
                       estCle <- a.estIdentifiant )
}
```

a) Écrire la règle qui transforme le `Modele` en `Schema`.

b) **R3.** Remplacer `Classe2Table` par deux règles *gardées* : l'une pour les classes qui ont un attribut identifiant, l'autre pour les classes qui n'en ont pas (création d'une colonne `id BIGINT` clé). Indice : une garde s'écrit `from c : Pim!Classe (condition)`.

c) **R4.** Écrire la règle `Association2CleEtrangere` qui crée la colonne `<de>_id` et la `CleEtrangere`. Comment rattacher ces éléments à la table de la classe `vers` ? (On suppose que la classe `de` n'a pas d'identifiant métier.)

d) Comparer avec le programme Java du cours : avantages et inconvénients de l'approche par modélisation.